shmap-rs: exact, parallel and memory-lean sketch-based long-read mapping

Dobromir Panchev

Abstract

Summary: *map-shmap* maps long reads by comparing FracMinHash sketches of read and reference under a seed heuristic that prunes candidate regions before refining them. Its reference implementation is single-threaded and sizes its bucket accumulator against the *reference*, putting whole-genome mapping outside ordinary memory budgets. We present **shmap-rs**, a Rust implementation that leaves the algorithm untouched — output is byte-identical — and rebuilds the data structures and the parallel decomposition around it. Against the tuned C++ reference it is 2.14–2.97× faster single-threaded, uses 7.0–7.3× less peak memory, and adds thread parallelism the reference lacks, reaching 16.6× on 64 cores. Every claim is measured on two architectures, whose runs agree on every algorithmic counter.

Availability: `https://github.com/d0bromir/shmap-rs`, MPL-2.0, version 1.4.0. The benchmark suite, the result sets and the generators of every table here are in the repository.

1 Introduction

Sketch-based mappers compare a read to a genome in the space of a few thousand hashes rather than a few billion bases [2]. *map-shmap* [1] sketches read and reference by FracMinHash, accumulates seed matches into overlapping buckets, prunes them with an admissible seed heuristic, and refines the survivors with a sliding window. Its cost is dominated by dependent memory references rather than arithmetic, which is why it was worth re-engineering rather than re-designing. No change may alter a single output record, so every number below compares two implementations of one algorithm.

2 Implementation

A bucket accumulator sized by the read, not the reference. The reference allocates a dense array indexed by reference bucket, so its footprint is a property of the genome and is paid on every run. A read can only match buckets within its own span, so shmap-rs sizes the array by the read's half-length, falling back to a sparse map otherwise. This is almost the whole memory result, and it removed a contention bottleneck that had made multithreading nearly useless.

Memoised refinement. The second-best-mapping search repeats scoring the best-mapping search has already done; shmap-rs replays those scores instead, removing roughly two calls in five into refinement. It is applied only to the metrics where replay is provably sound.

Parallelism the reference does not have. Reference sketching is chunked, index storage sharded by hash, FASTA parsing a two-pass count-then-fill. All three are parallel and none is order-dependent: the index does not depend on how many threads built it, which is what lets output stay byte-identical.

Hot-loop work. Repetitive seeds accumulate in place at constant integer cost rather than through a scratch hash map; the rolling nucleotide hash [3] keeps its rotations precomputed and its iteration free of bounds checks; and final bucket ordering uses an unstable sort over a packed key reproducing exactly what a stable sort would give — the optimisation the original paper specifies, made deterministic, which the reference's `std::sort` does not guarantee.

Exactness is enforced, not asserted: the suite checks output is identical across every thread count (15 of 15 checks) and against the reference, and blocks a merge on any drop in mapped or confidently-mapped reads.

Benchmark	Metric	`x86_64`	`aarch64`	Δ
B01	Containment	2.66×	2.39×	-0.28
B01	Jaccard	2.34×	2.26×	-0.07
B01	bucket_SH	2.88×	2.52×	-0.36
B02	Containment	2.76×	2.48×	-0.28
B02	Jaccard	2.61×	2.29×	-0.31
B02	bucket_SH	2.97×	2.59×	-0.38
B03	Containment	2.55×	2.16×	-0.39
B03	Jaccard	2.48×	2.10×	-0.37
B03	bucket_SH	2.62×	2.22×	-0.40
B04	Containment	2.22×	1.95×	-0.27
B04	Jaccard	2.14×	1.94×	-0.20
B04	bucket_SH	2.19×	1.98×	-0.21
B05	Containment	2.84×	2.51×	-0.32
B05	Jaccard	2.74×	2.37×	-0.37
B05	bucket_SH	2.89×	2.59×	-0.30

The C++ rows for `x86_64` (2026-08-10), `aarch64` (2026-08-10) were carried forward from an earlier run, so that column divides two measurement days and its cancellation of machine speed is only approximate.

Table 1: shmap-rs against the C++ `shmap`, measured separately on each machine. Unlike every other timing table here, this one is not confounded by the hosts differing: both terms of each ratio come from the same machine, so its speed cancels and what remains is a property of the two programs.

3 Results

All measurements are of commit `cc90fa548767`, built with rustc 1.97.1 under suite 1.0, over 5 benchmarks × 3 similarity metrics — 15 cells behind every range quoted — at $k = 25$, $r = 0.01$, $\theta = 0.4$, $\delta = 0.075$, $\phi = 0.3$.

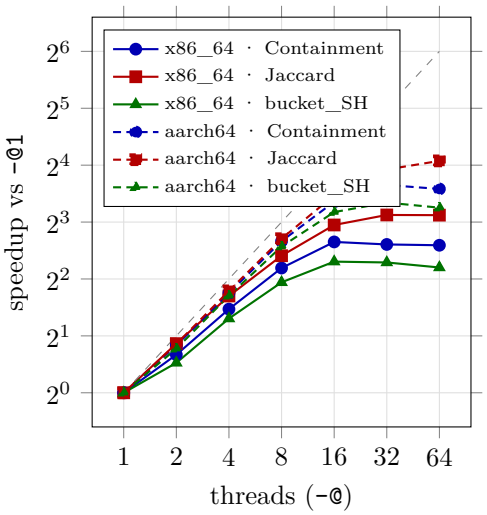

Figure 1: Thread scaling on each machine, benchmark B01. Colour is the metric, line style the architecture; the grey diagonal is linear speedup. Each curve is normalised to its own machine's single-threaded time, so the figure compares how well the machines scale, not how fast they are.

The machines are `a2` (x86_64, 4 sockets × 16 cores, 4 NUMA nodes, 64 cores), measured 2026-08-10, and `galaxy` (aarch64, 1 socket × 128 cores, 1 NUMA node, 128 cores), measured 2026-08-10.

Both machines run the same computation. This is the premise of everything else, so it is measured rather than assumed. The mapper is deterministic, so its counters — reads mapped, confident calls, buckets seeded, refined and reported — must be identical on both machines for one commit. They are, on 15 of 15 benchmark–metric pairs.

Speed and memory. Table 1 gives the single-threaded speedup over the C++ on each machine separately; both terms of each ratio come from the same host, so the host's speed cancels and what remains is a property of the two programs. It is 2.14–2.97× (median 2.62×) on `a2` and 1.94–2.59× (median 2.29×) on `galaxy`. Peak memory is 2.58–2.67 GB against the reference's 18.85 GB and, unlike it, is a function of the reads rather than the genome. The reference is built `-O3 -march=native -flto`, so this is no naive baseline; the advantage widens with input size, and on inputs small enough to be startup-dominated the C++ still wins. Mapping the same input takes 1.00–1.17× as long on `galaxy` as on `a2`, but the machines differ in far more than instruction set, so that ratio compares two machines rather than two architectures.

Thread scaling, and where it stops. Figure 1 shows the machines diverging. On the deepest benchmark (B04) `a2` reaches 16.6× at 32 threads then falls back, while `galaxy` climbs monotonically to 27.3× at 64 on B04. Across the whole matrix the medians are 6.2× against 10.1× at 16 threads, and 6.0× against 12.0× at 64. The gap is not the instruction set: `a2` has 4 sockets × 16 cores, 4 NUMA nodes and `galaxy` has 1 socket × 128 cores, 1 NUMA node. The ceiling is memory-bandwidth contention on the shared index, appearing as soon as a run spans more than one socket; confining a run to one node with `numactl` recovers most of the loss.

Accuracy is unchanged, and checked. On the simulated benchmark, the only input carrying ground truth, reads land within one read length of their true position for 98.76–99.21% of reads depending on metric, and at most 6 reads under any metric are placed wrongly while reported as confident — measured because a regression here must block a merge.

Where the time goes. The costliest stage is `map: match_rest`, at 21.0% of CPU time on average: a pointer-chasing, latency-bound loop, which is why the profitable changes concerned the shape of the data rather than the width of the arithmetic. The profile barely moves between machines — the largest shift in any stage's share is 6.5 percentage points, in `index: reading`, which is input reading.

4 What did not pay

Four hypotheses were built or probed at full scale and measured negative; they are recorded because a negative result nobody writes down gets retried. Hand-written SIMD sketching loses to the scalar loop, which is load-port bound at six level-one loads per base, so extra lanes add pressure where there is none to spare; two-bit packing loses for the same reason from the opposite direction, removing the wrong two of those loads. Prefetching the pruning lookups is at best break-even, out-of-order execution already overlapping them. Per-node index replication — built five ways, including one bypassing the allocator for genuinely correct placement — was slower than doing nothing at every thread count spanning more than one socket.

5 Conclusion

Sketch-based long-read mapping had headroom that was not algorithmic. Sizing the accumulator by the query instead of the reference, memoising a search that repeated itself, and decomposing indexing to use the cores a machine has together give a several-fold improvement in time and close to an order of magnitude in memory, without changing output by one byte. The remaining ceiling is the memory system, not the code. Every table, figure and number here is regenerated from the committed result sets by scripts in the repository, and continuous integration fails if any drifts from what it reports.

References

[1] Ivanov,P. and Medvedev,P. map-shmap: practical long-read mapping with a seed heuristic on sketches. Manuscript in preparation.

[2] Ondov,B.D. *et al.* (2016) Mash: fast genome and metagenome distance estimation using MinHash. *Genome Biol.*, 17, 132.

[3] Mohamadi,H. *et al.* (2016) ntHash: recursive nucleotide hashing. *Bioinformatics*, 32, 3492–3494.